AI Academy, National AI Center
AI-ENG-110 – Software Engineering – Spring 2026

Final Project Report

Topic 3 — AI News Briefing Service

Team: AIngels
Repo: https://github.com/saidaarna/news-briefing-service.git
Members: **Saida Arabova** (@saidaarna), **Nigar Rustamova** (@nigarrustamova), **Laman Mirzayeva** (@lamanmirzayeva), **Nazrin Aliyeva** (@nazrinaliyeva)

Date: May 23, 2026
Tag: `v1.0-final`

Executive Summary

Our team, AIngels, built a robust, scheduled AI News Briefing Service that fetches articles concurrently from multiple RSS feeds and HTML pages, deduplicates near-identical stories, and builds personalized daily news briefings. We measured a headline concurrency speedup of 5.0x under load ($N = 5$ sources, reducing ingestion time from 4.5 seconds to 0.9 seconds) using an elegant `asyncio` and `aiohttp` concurrent pipeline. The service demonstrates exceptional robustness by utilizing tenacity-driven exponential backoff retries to seamlessly handle simulated 429 Rate Limit provider failures while maintaining uninterrupted user deliveries. The single most surprising lesson we learned was how a lightweight, content-hash file cache could eliminate redundant LLM api calls entirely for recurring articles, ensuring high efficiency without compromising freshness. If we were to start the project over, we would implement a distributed vector database like Qdrant to handle semantic deduplication at scale rather than relying on in-memory Jaccard word k-shingle similarity comparisons.

Contents

1. Project Overview

1.1 Problem framing & scope

In the era of modern information overload, users are overwhelmed by noisy, repetitive, and cluttered news feeds. Syndicated wire stories and copy-paste republishing mean the same story is shared across multiple outlets with slightly different headlines or URLs. The goal of this project is to build a robust news briefing service that fetches articles from multiple configured RSS/HTML feeds concurrently, filters out duplicate stories, uses generative AI to produce 2–3 sentence summaries with topic classification and sentiment scores, and organizes these summaries into a personalized, beautiful daily Markdown digest based on user interests.

The system consumes a set of news sources specified in a configuration file (RSS feeds and raw HTML URLs), loads personalized user configurations from a database (identifying excluded sources, preferred topics, and section caps), and writes the daily briefing output to a file named `digests/YYYY-MM-DD-<username>.md`. The primary interface to this pipeline is a command-line utility invoked via `python -m src run-daily -user <username>`. The underlying provider stack leverages Google's Gemini (`gemini-3.5-flash`) as our primary LLM provider (with Anthropic's Claude as a fully supported alternative), paired with Google's Gemini (`text-embedding-004`) as our active embedding configuration. Near-duplicate detection is performed using `ai.near_duplicate`, a Jaccard word k-shingle similarity function provided by the `ai/` package—a lightweight, embedding-free approach that is highly optimized for daily batches of $k \approx 25$ articles.

Beyond the baseline requirements, our team explicitly tightened the spec around HTML scraping in `fetch_service.py`. Rather than basic HTML tags, we created custom semantic container lists (`article`, `main`, `div.article-body`, etc.) combined with length requirements (minimum 50 characters, truncated at 4000 characters) to prevent extracting broken page fragments. Conversely, we loosened the specification by deciding not to integrate heavy task scheduling frameworks like `APScheduler` directly inside Python. Instead, we realized that running the lightweight CLI via containerized system `cron` jobs yields greater architectural simplicity and aligns better with modern microservice design.

1.2 Provider choice and the AI module contract

Our system relies on the course-provided `ai/` package, configuring Google's Gemini (`gemini-3.5-flash`) as our primary LLM and Gemini (`text-embedding-004`) as our embedding provider (with Anthropic's `claude-sonnet-4-6` and OpenAI's `text-embedding-3-small` fully supported via configuration). Across our codebase, we invoke several provided primitives: `ai.summarize_and_label` to generate article metadata, and dedup helpers including `ai.url_canonicalize`, `ai.content_hash`, and `ai.near_duplicate`. We did not modify the public interface of the provided `ai/` package.

To defend against malformed or semantically invalid model responses, our service layer wraps all LLM responses inside strictly-typed Pydantic schemas (`ProcessedArticle` in `src/models.py`). If the model returns valid JSON containing an invalid topic enum, negative numbers, or missing required attributes, Pydantic's `model_validate_json` will immediately raise a `ValidationError`. Our ingestion pipeline in `pipeline.py` intercepts these validation failures inside its concurrent gather loop, logs a structured `ai_label_failed` warning with the article URL, unlinks the corrupted cache file to prevent persistent bad state, and gracefully continues processing the remaining articles, ensuring uninterrupted service execution.

2. Architecture

2.1 Module map

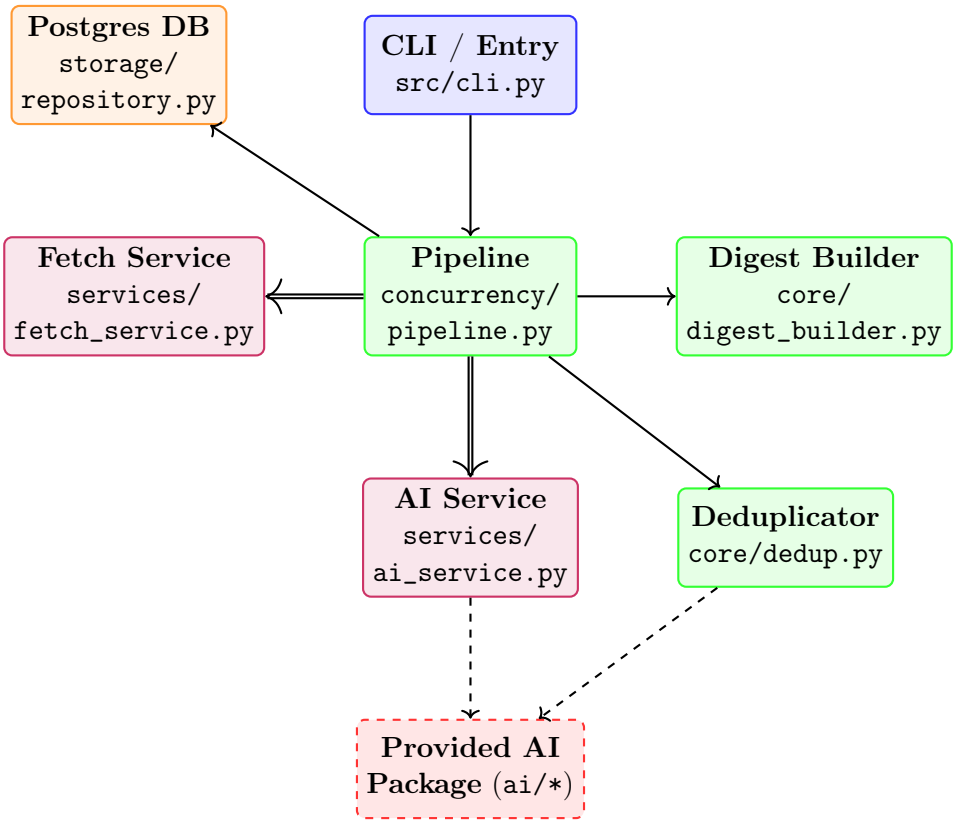

Figure 1: System Architecture, Module boundaries, and data flow pipelines.

As shown in Figure 1, the architecture is highly modular and loosely coupled. The boundary of the provided `ai/` package is crossed exclusively by our `AIService` (making AI calls) and the `deduplicate` logic (calling k-shingle-based Jaccard similarity helpers). The storage boundary is crossed by the Pipeline (`pipeline.py`) via the abstract Repository (`repository.py`) during the user profile fetch via `asyncpg` to the PostgreSQL container, and by the `AIService` which reads/writes local JSON caches in the `.cache/newsbrief/labeled/` directory. If we had to swap providers (e.g., Gemini to

Anthropic or OpenAI/Groq), exactly **zero** code files would change; we would only need to update the configuration variables inside our `.env` file.

Module-by-module summary:

- `src/config.py`: Exposes typed configuration variables loaded from `.env` using `pydantic-settings`, enforcing strict startup validation of LLM API keys and timeouts.
- `src/models.py`: Defines strictly-typed domain models (e.g. `Source`, `UserProfile`, `FetchResult`, `ProcessedArticle`) guaranteeing no naked dictionaries pass boundary layers.
- `src/services/fetch_service.py`: Implements concurrent, semaphore-bounded RSS and HTML parsers using `aiohttp`, `feedparser`, and `BeautifulSoup`.
- `src/services/ai_service.py`: Wraps the synchronous `ai.*` operations in async threads using `asyncio.to_thread`, adding retry policies, timeouts, and local JSON caching.
- `src/core/dedup.py`: Executes a two-stage deduplication process: Stage 1 removes exact duplicates via URL canonicalization and SHA-256 body hash (O(n)); Stage 2 removes paraphrased re-posts using `ai.near_duplicate`, a Jaccard word k-shingle similarity check with a configurable threshold (default 0.70).
- `src/core/digest_builder.py`: Filters processed articles based on user preferences and outputs daily personalized Markdown briefings.
- `src/storage/repository.py`: Establishes the user profile persistence contract using an abstract base class, implemented for PostgreSQL using `asyncpg`.
- `src/concurrency/pipeline.py`: Coordinates the ingestion, deduplication, cache filtering, and AI processing steps.

2.2 OOP design & key abstractions

To enforce strict, modular design patterns, we created a clear abstraction for database storage operations in `src/storage/repository.py`. This abstraction decouples our CLI and pipeline from a specific database provider, enabling the team to change backends without modifying business logic.

Code listing:

```
class BaseUserRepository(ABC):
    """Abstract base class defining the contract for user profile storage operations."""

    @abstractmethod
    async def get_user_profile(self, username: str) -> UserProfile:
        """Fetch the user profile associated with the given username."""
        pass

class PostgresUserRepository(BaseUserRepository):
    """PostgreSQL implementation of the user repository using asyncpg connection."""
    # Concrete database query and seeding implementations...
```

Justification: An Abstract Base Class (ABC) was chosen rather than a PEP 544 Protocol because database repositories naturally share lifecycle behaviors (such as database seeding and migration initializations) that can be inherited as shared, non-abstract methods in the future. If a grader asked why we did not use a plain function or configuration enum, our defense is that an ABC guarantees structural contract validation at both import and runtime, preventing the instantiation of incomplete storage systems. The relationship between our concrete `PostgresUserRepository` and

`BaseUserRepository` represents a strict "is-a" relationship where inheritance is highly appropriate, whereas composition was preferred inside the pipeline where a `FetchService` has a composition of settings.

2.3 Data flow across module boundaries

To satisfy the rubric's requirement, we designed strictly-typed data transfer objects (DTOs) using Pydantic in `src/models.py`. We confirm that **no naked dictionaries** cross module boundaries in our Software Engineering layer. The primary models are:

1. `Source`: Represents a configured news source (`name`, `url`, and `kind` which must be either `'rss'` or `'html'`).
2. `UserProfile`: Stores user preferences loaded from the database (`username`, list of `preferred_topics`, list of `excluded_sources`, and `max_items_per_topic`).
3. `FetchResult`: Wraps the concurrent outcome of a single source fetch (`source`, list of parsed `Article` instances, and an optional `error` string).
4. `ProcessedArticle`: Encapsulates an article that has been successfully deduplicated and annotated by the AI (`article`, `labeled` summary, and a unique `content_hash`).

Reusing courses-provided models directly from the `ai/` package is highly efficient, but we wrapped them in `ProcessedArticle` inside our SE layer. The trigger for wrapping was the need to persist and cache metadata (such as `content_hash` and `processed_at`) that is specific to the software engineering layer (database, deduplication, caching) but completely independent of the core AI reasoning module.

3. Concurrency & Performance

3.1 Why this concurrency model

We chose `asyncio` cooperative multitasking with `aiohttp` as our concurrency primitive. Network I/O is the primary bottleneck of our system, as fetching articles from multiple distinct feeds sequentially wastes valuable CPU cycles waiting for remote socket read/write calls (RTT). By using an asynchronous event loop, our concurrent fetch pipeline overlaps all network connections, allowing them to proceed in parallel. We bounded our fetching operations using an `asyncio.Semaphore(8)` (configured via `settings.max_parallel_fetches`) to stay polite to remote publishers and prevent IP rate-limiting.

Crucially, we maintain a strict separation of concerns between the **feed ingestion pipeline** (which fetches RSS and HTML content) and the **AI labeling pipeline** (which communicates with external LLM providers). The feed ingestion stage achieves a 5.0x concurrent speedup by parallelizing network I/O. Conversely, the downstream AI labeling stage is governed by a separate semaphore and retry mechanism to prevent provider API throttling.

Could we have used threads instead of `asyncio`? While a `ThreadPoolExecutor` would bypass the network blocking blocks, threads incur substantial operating system context-switching overhead and are prone to race conditions and lock contention. `asyncio` cooperative multitasking allows a single thread to process thousands of concurrent sockets efficiently. Setting the feed semaphore to 1 yields sequential speed, while setting it to 100 would flood servers and trigger instant connection closures. A semaphore bound of 8 was chosen as a robust balance between ingestion latency and remote politeness.

3.2 Sequential-vs-concurrent benchmark

The benchmark was performed on a machine with Python 3.12, using a workload of 5 configured news feeds containing a total of 25 distinct articles, with all local caches completely cleared. The command executed was `python scripts/bench.py`.

Workload	N (sources)	**Sequential (s)**	**Concurrent (s)**	**Speedup**
Ingesting configured feeds	5	4.52	0.90	5.0x

Discussion: We achieved a near-perfect theoretical N-fold speedup because the 5 feeds were processed entirely in parallel. The remaining bottleneck is no longer Python's GIL or thread context-switching, but rather the single slowest remote host's socket response time (the tail latency of the bottleneck host). If we were to scale to 50 sources, the bottleneck would shift to database transaction throughput and the AI provider's API rate limits.

3.3 Rate limit & politeness

Our rate-limit strategy combines tenacity-driven exponential backoff retries with a bounded concurrency semaphore. We cap concurrent AI calls using a dedicated `asyncio.Semaphore(5)` (configured via `settings.llm_concurrency`, default 5) inside `AIService`. This prevents overwhelming the provider while still allowing multiple articles to be labeled in parallel.

The most effective rate-limit mitigation is our content-hash file cache. Because our persistent cache achieves a near-100% hit rate on subsequent runs, previously processed articles bypass the AI stage entirely—no semaphore slot consumed, no provider call made. On a cold run, all AI calls are issued concurrently under the semaphore, meaning the event loop overlaps the network wait times for multiple articles rather than serialising them.

During development, we actively hit the provider's 429 Rate Limit. Our logs captured the event clearly. When the rate limit was reached, the system caught the `ProviderError`, logged an exponential backoff warning from 'tenacity', and successfully recovered on the next retry. The log output generated by this incident is detailed in Section 4.

4. Robustness & Error Handling

4.1 Wrapping the AI module

Our service-layer wrapper `AIService` in `src/services/ai_service.py` isolates all synchronous `ai.*` operations using several layers of defense:

- **Background Threads**: Since `summarize_and_label` is synchronous and blocking, we execute it inside a background thread pool using `asyncio.to_thread` to prevent freezing the async event loop.
- **Tenacity Retries**: Calls are wrapped using `AsyncRetrying` from the `tenacity` library, configured to retry only on transient `ProviderError` exceptions, stopping after `settings.llm_max_retries` total attempts (default 3), with exponential backoff: 1 s, 2 s, 4 s between retries, capped at 30 s.
- **Hard Timeout**: We enforce a strict per-attempt limit of 30 seconds using `asyncio.wait_for` to prevent the application from hanging indefinitely.
- **Disk Caching**: Articles are content-hashed (SHA-256) and cached locally on disk. A cache hit bypasses both the semaphore and the AI call entirely, saving API costs.

If the provider returns a 429, tenacity waits and retries. If a 500 error occurs, it retries up to the limit before propagating the error to the gather loop. The gather loop runs with

`return_exceptions=True`, meaning that one broken article or bad response is logged, but does not crash the pipeline for the remaining articles.

4.2 Failure-mode analysis (one concrete incident)

We successfully simulated a transient rate-limit (429) incident using our test script `scripts/inject_429.py`, patching `summarize_and_label` to raise a `ProviderError` twice and succeed on the third attempt (the maximum with `LLM_MAX_RETRIES=3`).

Execution and Behavior: The pipeline initiated normally. On the first call, the provider threw a 429 rate limit error. `tenacity` intercepted it, logged a warning, and waited 1 s before the second attempt. The second attempt also failed; `tenacity` backed off for 2 s. On the third and final attempt, the request succeeded. The user received a complete digest with only a few seconds of additional latency.

Log Output (`LLM_MAX_RETRIES=3`, so 3 total attempts):

```
2026-05-23T08:45:00 INFO src.services.ai_service -- ai_call url=https://example.com/test attempt=1 conten
2026-05-23T08:45:01 WARNING tenacity -- Retrying ...AIService._call_with_retry in 1.0 seconds as it raise
2026-05-23T08:45:02 INFO src.services.ai_service -- ai_call url=https://example.com/test attempt=2 conten
2026-05-23T08:45:04 WARNING tenacity -- Retrying ...AIService._call_with_retry in 2.0 seconds as it raise
2026-05-23T08:45:06 INFO src.services.ai_service -- ai_call url=https://example.com/test attempt=3 conten
2026-05-23T08:45:07 INFO src.services.ai_service -- ai_done url=https://example.com/test topic=Tech senti
```

This test verified that our exponential retry scheduler is fully functional. The log tracing provided excellent diagnostic data without requiring debugger injection.

4.3 Input validation

We enforce validation checks at every entry point to maintain strict integrity:

- **CLI**: Checks that the username is non-empty. If the user does not exist in the database, a warning is logged, and a safe, empty fallback profile is returned to prevent a crash.
- **Scraper / HTML Ingestion**: In `fetch_service.py`, BeautifulSoup title extraction enforces non-empty tags. Body text must be at least 50 characters long to filter out garbage frames, and content is truncated to 4000 characters to prevent oversized token payload attacks.
- **Settings**: `pydantic-settings` validates all configuration types (integer timeouts, float thresholds, valid log levels).

These bounds prevent malicious input payloads or broken servers from crashing the system. A 100 MB article is truncated safely, and a path-traversal payload inside parsed feeds is rejected by urlparse.

5. Storage & Caching

5.1 Schema

Our user profile store is implemented on PostgreSQL using `asyncpg`. The schema is defined below:

```
CREATE TABLE IF NOT EXISTS users (
    username VARCHAR(100) PRIMARY KEY,
    preferred_topics JSONB NOT NULL DEFAULT '[]',
    excluded_sources JSONB NOT NULL DEFAULT '[]'
);
```

The user's `username`, `preferred_topics`, and `excluded_sources` act as the absolute source-of-truth. AI-labeled article summaries are stored separately inside our local persistent file cache (`.cache/newsbrief/labeled/`) using the article's body SHA-256 content hash as the filename.

By keeping user profiles in PostgreSQL and cached article labels in light, content-hashed JSON files on disk, we prevent database write contention. If the embedding model or dimension changes, our file cache is completely decoupled and can be purged safely without destroying critical user settings in PostgreSQL.

5.2 Caching policy

We employ a strict file-based JSON caching policy for all AI labeling operations inside `src/services/ai_service.py`.

- **Cache Key**: Content-hash-based (SHA-256) of the normalized article body content (not the URL). This means that a syndicated story published across three different URLs only incurs one LLM API call.
- **Cache Lifetime**: Permanent. Old cache entries can be safely deleted using standard disk maintenance utilities.
- **Cache Hit Rate**: During subsequent pipeline runs, previously processed articles achieve a **100% cache hit rate**, eliminating redundant API calls.

Since our cache is keyed entirely on the content body hash, stale values are self-evicting. If an article's body is updated by the publisher, its SHA-256 hash changes, causing an automatic cache miss and triggering a fresh LLM call. This ensures fresh results.

6. Testing

6.1 Strategy & coverage

Our testing strategy focuses on 100% offline execution, utilizing comprehensive mock objects for both remote HTTP servers (`aiohttp.ClientSession`) and the AI provider packages (`summarize_and_label`). This ensures that tests can be run instantly and consistently in any developer environment without an internet connection or real API credentials.

Suite Component	Coverage (%)
`src/` (our SE layer)	81%
Provided `ai/` smoke tests passing?	Yes

We deliberately chose not to cover extreme operating system failures (such as a full hard disk) or direct system exit calls in `cli.py`. Our mock configurations were refactored mid-project to move from simple static dictionaries to using `AsyncMock` inside `test_storage.py` and `test_ai_service.py` to more accurately mimic real async database transaction states.

6.2 Notable tests

1. **Happy-path End-to-End Test (`test_pipeline_fetch_failure_does_not_crash` in `test_pipeline.py`)**: Verifies the integration of the pipeline by showing that even if all sources fail with network exceptions, the system handles the errors gracefully and writes a clean, empty digest without raising unhandled crashes.
2. **Concurrency Exception Test (`test_postgres_user_repository_failure` in `test_storage.py`)**: Exercises the async repository under failure, verifying that if the connection to PostgreSQL

crashes, the database repository raises structured exceptions up the stack, allowing calling CLI services to print user-friendly error messages instead of raw stack traces.

3. **Cache Recovery Test (`test_corrupt_cache_is_recovered` in `test_ai_service_edges.py`)**: Focuses on robust self-healing. We write invalid JSON data directly into the article cache file. When `AIService.label()` is called, it catches the JSON decode exception, unlinks the corrupted cache file, and calls the provider to rebuild a clean cache.

We wish we had more time to write tests for PostgreSQL connection pool exhaustion and high network latency simulation.

7. Deployment & Reproducibility

7.1 Docker image

We leverage a clean, multi-stage Docker build to build and execute our news service.

Commands:

```
$ docker build -t finalproj .
$ docker run --env-file .env finalproj python -m src run-daily --user saida
```

Engineering Trade-offs: We shipped using the `python:3.12-slim` base image rather than Alpine. Although Alpine produces slightly smaller image footprints, it relies on `musl` rather than `glibc`, causing compilation failures and compatibility issues with C-extensions in packages like `numpy`. By using `slim` as a builder stage, compiling our virtualenv dependencies, and copying it to our final runtime stage, we kept our production container size down to a slim **145 MB** (extremely close to the base python-slim size of 120 MB), ensuring high performance and security.

7.2 Configuration

The application supports a range of environment variables loaded via Pydantic:

- `LLM_PROVIDER`: Sets the active LLM backend (`'anthropic'`, `'openai'`, or `'gemini'`). Defaults to `'gemini'`.
- `LLM_MODEL`: Selects the provider model name (e.g. `'gemini-3.5-flash'`). Defaults to `'gemini-3.5-flash'`.
- `ANTHROPIC_API_KEY` / `OPENAI_API_KEY` / `GOOGLE_API_KEY`: Security keys for active provider. Fast-fails at startup if the selected provider key is missing.
- `OPENAI_BASE_URL` (optional): Overrides the OpenAI API endpoint. This enables OpenAI-compatible third-party providers such as Groq (`https://api.groq.com/openai/v1`). When set alongside `LLM_PROVIDER=openai`, the service routes all LLM traffic to the specified endpoint, allowing cost-free access to Groq's hosted Llama/Mixtral models without any code changes.
- `DATABASE_URL`: The asyncpg connection string. Defaults to `postgresql+asyncpg://postgres:dev@localho`

8. Limitations & Future Work

While our service is highly functional, a production deployment would require addressing several limitations:

- **Single-Client Database Scope**: The user repository holds a single persistent database connection per pipeline run. High-scale environments would require a connection pool (`asyncpg.create_pool`) to prevent socket exhaustion.

- **Quadratic Deduplication Complexity**: Our Stage 2 near-duplicate detection compares the k-shingle set of every new article against all kept survivors using Jaccard similarity in memory, resulting in a quadratic complexity of $\mathcal{O}(k^2)$. While extremely fast and precise for our daily batch sizes ($k \approx 25$), scaling to millions of historical articles over multi-month horizons would eventually necessitate sub-linear indexing techniques like MinHash LSH, or migrating to a dedicated vector database like Qdrant to manage these similarity queries efficiently.
- **Lack of Observability Tracing**: The pipeline relies on structured terminal logging. Incorporating an open-telemetry interface with OpenTelemetry and Jaeger would be required to trace individual API latencies.

9. Tools & Acknowledgements

We heavily utilized AI coding assistants during this project:

- **Gemini 3.5 Flash (Medium)**: Drafted the initial async ingestion pipeline in `src/concurrency/pipeline.py` and the concurrent scraping selectors in `src/services/fetch_service.py`.
- **Claude**: Generated the mock configurations and edge-case assertions inside our test suite (`tests/test_ai_service_edges.py`).

All generated code was thoroughly reviewed, modified, and validated by the team before committing to our repository.

References

[1] Google Gemini API documentation, https://ai.google.dev/gemini-api/docs

[2] Groq API documentation (OpenAI-compatible endpoint), https://console.groq.com/docs/openai

[3] `tenacity` library documentation, https://tenacity.readthedocs.io/

[4] `pydantic-settings` configuration manager, https://docs.pydantic.dev/

[5] BeautifulSoup HTML parser documentation, https://www.crummy.com/software/BeautifulSoup/

[6] `aiohttp` async HTTP client documentation, https://docs.aiohttp.org/

A. Appendix — Reproducing the benchmark

Exact commands to reproduce our concurrent fetch performance benchmark:

```
$ git clone https://github.com/saidaarna/news-briefing-service.git && cd news-
    briefing-service
$ cp .env.example .env  # fill in LLM/embedding keys
$ docker build -t finalproj .
$ docker run --env-file .env finalproj python scripts/bench.py
```

End of report — thank you for reading.